

React

Séance 7 - Introduction au front-end : l'interface rencontre l'API

Cours 420-941-MA - Collège de Maisonneuve - Groupe 25604

composants - JSX - props - state (useState) - useEffect - axios - auth côté client

Où on en est, où on va

Récap S3-S6 et objectifs S7

Acquis (S1 -> S6)

- Un BACKEND complet et sécurisé
- API REST (Express) + PostgreSQL (Prisma/Neon)
- Consommer une API externe (axios)
- Authentification JWT + rôles
- MAIS : on teste tout dans test.rest ...

Objectifs de la séance

- Comprendre le rôle d'un frontend / d'une SPA
- Créer une app React (Vite)
- Composants, JSX, props, state
- Afficher des listes, gérer des formulaires
- useEffect + axios : appeler NOTRE API
- Se connecter et afficher des données réelles

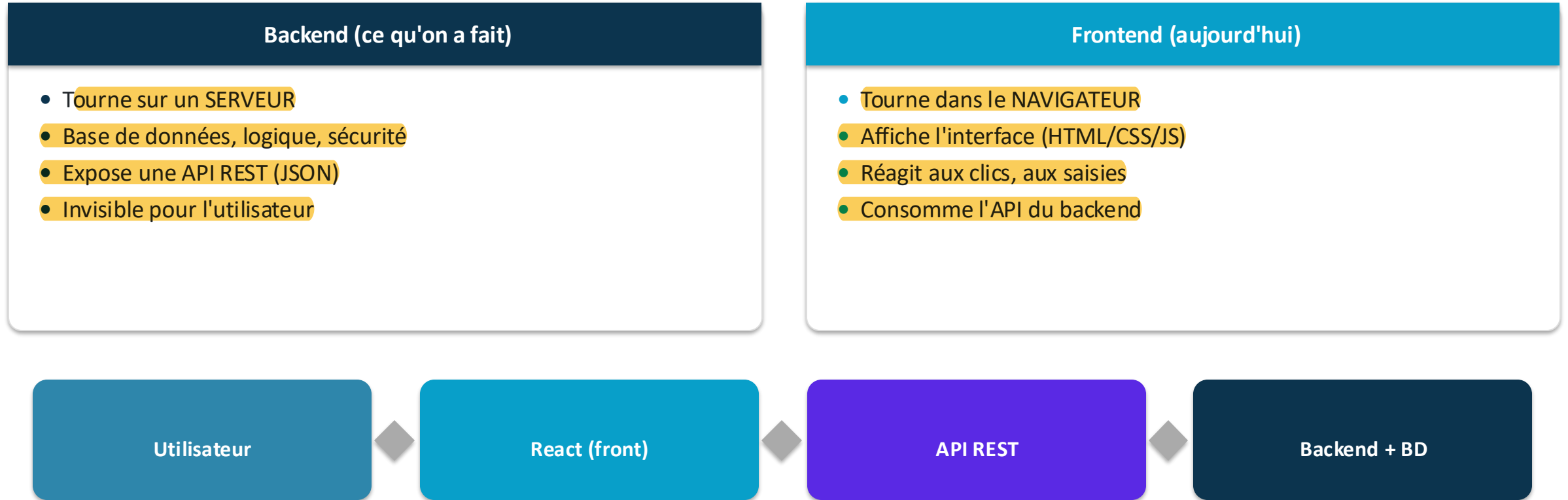
Le grand moment. Jusqu'ici, notre backend n'avait pas de visage : on l'interrogeait avec Postman. Aujourd'hui, on lui construit une INTERFACE. React va afficher, dans le navigateur, les données servies par notre propre API - exactement ce que verra un vrai utilisateur. Le front et le back se rencontrent enfin.

PARTIE 1

Pourquoi un frontend ? Pourquoi React ?

Frontend vs Backend

Deux moitiés d'une même application



Le frontend et le backend communiquent par des requêtes HTTP (les mêmes que dans Postman) : React demande des données, l'API répond en JSON, React les affiche.

React : une interface qui réagit

Single Page Application (SPA)

React est une librairie JavaScript pour construire des interfaces. Une page devient une SPA : le navigateur charge l'app une fois, puis React met à jour uniquement les parties de l'écran qui changent - sans recharger toute la page.

Composants

- L'interface = des briques
- réutilisables et imbriquées
- Un bouton, une carte, une liste

Déclaratif

- On décrit CE QU'ON VEUT voir
- selon l'état (state)
- React s'occupe du 'comment'

Réactif

- L'état change -> l'écran
- se met à jour tout seul
- Plus de manipulation manuelle

À retenir :

- React n'est pas 'magique' : c'est du JavaScript (et du TypeScript) organisé.
- On retrouve tout ce qu'on connaît : variables, fonctions, tableaux, .map(), async/await.
- Le plus utilisé au monde : énorme communauté, débouchés, base pour Next.js.

PARTIE 2

Créer et comprendre une app React

Créer l'application avec Vite

L'outil moderne pour démarrer une app React

1. Créer le projet (React + TypeScript) :

```
npm create vite@latest mon-front -- --template react-ts
cd mon-front
npm install
npm run dev          # -> http://localhost:5173
```

bash

2. Structure d'un projet Vite + React :

```
mon-front/
src/
  main.tsx      <- point d'entree
  App.tsx       <- composant racine
  components/   <- nos composants
  index.css
index.html
package.json
```

tree

Vite en 2 mots

- Démarrage quasi instantané
- Rechargement à chaud (HMR) :
- on sauvegarde -> l'écran
- se met à jour tout seul
- Remplace l'ancien Create-React-App

Le JSX : du HTML dans du JavaScript

Écrire l'interface directement dans le code

```
function App() {  
  const nom = 'Alice'  
  const heure = new Date().getHours()  
  
  return (  
    <div>  
      <h1>Bonjour {nom} !</h1>  
      {heure < 18 && <p>Bonne journee</p>}  
    </div>  
  )  
}
```

TSX

Règles du JSX

- Ça ressemble à du HTML...
- ...mais c'est du JavaScript
- { } = insérer du JS
- (variable, calcul, condition)
- className (pas class)
- Un seul élément racine

Le JSX est transformé en JavaScript par Vite. Les accolades { } sont la porte entre le HTML et le code : on y met une variable, un calcul, un .map()... Tout ce qui est entre accolades est évalué comme du JavaScript.

Composants et props

Des briques réutilisables, paramétrables

```
// Un composant = une fonction qui renvoie du JSX
function Carte({ titre, auteur }: {
  titre: string; auteur: string
}) {
  return (
    <div className='carte'>
      <h3>{titre}</h3>
      <p>par {auteur}</p>
    </div>
  )
}

// On le REUTILISE avec des props différentes
<Carte titre='Dune' auteur='Herbert' />
<Carte titre='1984' auteur='Orwell' />
```

TSX

Les props

- = les 'paramètres' d'un composant
-
- Passées comme des attributs HTML
-
- En LECTURE SEULE : un composant ne modifie pas ses props
-
- Typées avec TypeScript

Le state avec useState

La mémoire d'un composant : ce qui change dans le temps

```
import { useState } from 'react'

function Compteur() {
  // [valeur, fonction pour la changer]
  const [n, setN] = useState(0)

  return (
    <button onClick={() => setN(n + 1)}>
      Cliques : {n}
    </button>
  )
}
```

TSX

Idée clé

- useState crée une variable 'mémorisée'
-
- On NE modifie JAMAIS la valeur directement
- -> on appelle setN(...)
-
- Quand le state change, React RE-AFFICHE le composant tout seul

Différence props / state : les props VIENNENT du parent (lecture seule) ; le state APPARTIENT au composant et peut changer (via son setter).

Afficher une liste avec .map()

Transformer un tableau de données en tableau de JSX

```
const livres = [
  { id: 1, titre: 'Dune' },
  { id: 2, titre: '1984' },
]

return (
  <ul>
    {livres.map((l) => (
      <li key={l.id}>{l.titre}</li>
    ))}
  </ul>
)
```

TSX

Le .map() du JS

- On connaît déjà .map()
- (transformer un tableau)
-
- Ici : chaque donnée
- devient un élément JSX
-
- key={...} : un identifiant
- UNIQUE par élément
- (React en a besoin pour
- suivre les changements)

Un formulaire contrôlé

Le state suit ce que l'utilisateur tape

```
function Recherche() {  
  const [texte, setTexte] = useState('')  
  
  return (  
    <form onSubmit={(e) => {  
      e.preventDefault()           // pas de rechargement  
      console.log('Recherche :', texte)  
    }}>  
      <input  
        value={texte}  
        onChange={(e) => setTexte(e.target.value)}  
      />  
      <button>Chercher</button>  
    </form>  
  )  
}
```

TSX

Contrôlé = ?

- value vient du state
- onChange met à jour le state
- -> le state est la 'source de vérité'
- e.preventDefault()
- empêche le rechargement de la page (défaut HTML)

PARTIE 3

Brancher React à NOTRE API

useEffect + axios : charger des données

Aller chercher les données au démarrage du composant

```
import { useState, useEffect } from 'react'
import axios from 'axios'

function ListeLivres() {
  const [livres, setLivres] = useState([])

  // s'exécute UNE FOIS, au montage du composant
  useEffect(() => {
    axios.get('http://localhost:3000/livres')
      .then(res => setLivres(res.data))
  }, []) // [] = une seule fois

  return <ul>{livres.map(l:any =>
    <li key={l.id}>{l.titre}</li>)}</ul>
}
```

TSX

Le trio gagnant

- useState : stocker
- les données reçues
-
- useEffect : lancer
- l'appel au démarrage
-
- axios : parler à
- NOTRE API
-
- [] = ne s'exécute
- qu'une seule fois

Chargement et erreurs

Un vrai front gère les 3 états d'un appel réseau

```
const [livres, setLivres] = useState([])
const [chargement, setChargement] = useState(true)
const [erreur, setErreur] = useState('')

useEffect(() => {
  axios.get('http://localhost:3000/livres')
    .then((res) => setLivres(res.data))
    .catch(() => setErreur('Echec du chargement'))
    .finally(() => setChargement(false))
}, [])

if (chargement) return <p>Chargement...</p>
if (erreur) return <p>{erreur}</p>
```

TSX

3 états à gérer

- 1. En cours
 - -> 'Chargement...'
 -
- 2. Erreur
 - -> message clair
 -
- 3. Succès
 - -> afficher les données
 -
 - L'utilisateur n'est
 - jamais devant un écran
 - vide sans explication

Le piège CORS

Front (5173) et back (3000) sont sur des origines différentes

Le navigateur bloque par sécurité les requêtes entre origines différentes (localhost:5173 -> localhost:3000). Symptôme : une erreur 'CORS policy' dans la console. La solution se règle CÔTÉ BACKEND.

Dans le backend Express (une seule fois) :

```
// terminal du BACKEND
npm install cors

// src/server.ts
import cors from 'cors'
app.use(cors()) // autorise le front a appeler l'API
```

TS

À retenir

- CORS = Cross-Origin Resource Sharing. C'est une protection du navigateur, pas un bug.
- Ça se corrige côté serveur (le backend décide qui a le droit de l'appeler).

Se connecter depuis React (JWT)

Récupérer le token, le stocker, l'envoyer

```
// 1) Login : on reçoit un token et on le garde
async function connexion(email: string, mdp: string) {
  const { data } = await axios.post(
    'http://localhost:3000/auth/login',
    { email, password: mdp })
  localStorage.setItem('token', data.token)
}

// 2) Appeler une route protegee avec le token
const token = localStorage.getItem('token')
await axios.get('http://localhost:3000/mes-suivis', {
  headers: { Authorization: `Bearer ${token}` },
})
```

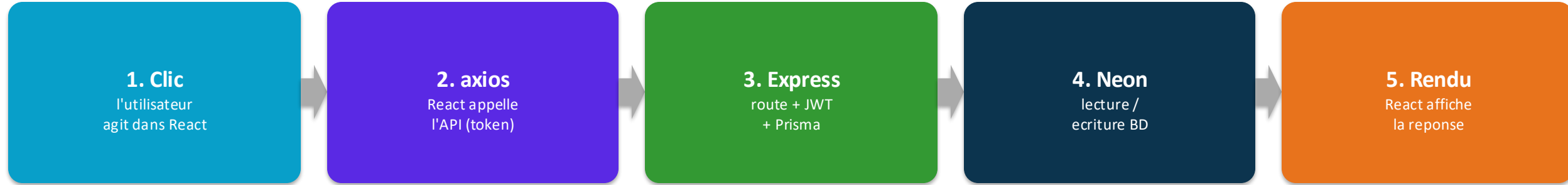
TSX

Le cycle JWT

- Login -> token
- On stocke le token
- (localStorage)
- On l'envoie dans
- le header
- Authorization
- Le backend vérifie
- (vu en S6)

Le tableau complet : full-stack

Ce que vous savez faire, du clic à la base de données



Vous êtes maintenant développeur FULL-STACK.

Backend (Express, Prisma, Neon, JWT) + Frontend (React) qui dialoguent via une API. C'est l'architecture de la quasi-totalité des applications web modernes.

À vous de jouer

Un mini-front pour votre API

- 1. **Créer l'app** : `npm create vite@latest mon-front -- --template react-ts`
- 2. **Installer axios** dans le front (`npm install axios`)
- 3. **Activer cors** dans votre backend (`npm install cors + app.use(cors())`)
- 4. **Afficher une liste** de votre API (`useEffect + axios + .map()`)
- 5. **Gérer** les états chargement / erreur / succès
- 6. **Bonus** : un formulaire de connexion qui stocke le token et affiche des données protégées

Objectif : voir, dans le navigateur, de VRAIES données servies par votre propre backend.

Récapitulatif de la séance

Ce qu'il faut retenir de React

Les bases

- Vite : créer l'app
- JSX : HTML dans le JS
- Composants + props
- state : useState
- Listes : .map() + key

Interactivité

- onClick / onChange
- Formulaires contrôlés
- e.preventDefault()
- props (parent -> enfant)
- state (mémoire locale)

Données & API

- useEffect (au montage)
- axios vers NOTRE API
- chargement / erreur
- JWT : token + header
- CORS (côté backend)

La commande à mémoriser :

```
npm create vite@latest mon-front -- --template react-ts
npm install axios
# et cote backend : npm install cors -> app.use(cors())
```

bash

Pour la prochaine séance

On consolide et on prépare la suite

À faire de votre côté

- Créer un front React (Vite)
- Activer cors dans le backend
- Afficher une liste depuis l'API
- Gérer chargement / erreur
- Un formulaire de connexion (token)
- Pousser le tout sur GitHub

Au programme ensuite

- Composants réutilisables + CSS
- Le routing (plusieurs pages)
- Contexte / état global (auth)
- Formulaires avancés
- Vers Next.js (full-stack)
- Déploiement (Vercel)

La boucle est bouclée : votre application a enfin une interface.

Backend sécurisé + Frontend React = une vraie application web full-stack.

Questions ?

Prochaine étape : donner une interface à votre API.

Créez, affichez, connectez. Bienvenue dans le full-stack.